Floriend Vermond: Base de datos para un sistema de E-Commerce de Ropa

Informe Técnico

Carlos Steven Avilés Roque

Allan Fauricio Fonseca Batista

Jimena Rivera Sancho

Isaac Andrey Sánchez Delgado

Profesor: Michael Robles Sánchez

UNIVERSIDAD FIDÉLITAS

Facultad de Ingeniería en Sistemas de Computación

SC-503: Administración de Base de Datos

I Cuatrimestre, 2026

Resumen

El presente proyecto tiene como objetivo diseñar e implementar una base de datos relacional en Oracle para la empresa Floriend Vermond. Esta es una tienda de comercio electrónico dedicada a la venta de ropa formal y casual. A partir del análisis inicial se identificaron problemas en la gestión de inventarios, el seguimiento de pedidos y la administración de accesos. Estas situaciones generaban inconsistencias en la información y limitaban el crecimiento del negocio.

Para solucionar estos problemas se desarrolló un sistema basado en Oracle Database 21c XE. Se incorporaron esquemas organizados, tablas dimensionales y transaccionales, restricciones de integridad, auditoría, encriptación de datos, índices de optimización, vistas materializadas y un plan de respaldos mediante RMAN. Además, se implementaron controles de seguridad basados en roles y perfiles siguiendo el principio de mínimo privilegio.

Como resultado se logró centralizar la información, mejorar la consistencia de los datos, optimizar el rendimiento de las consultas y fortalecer la seguridad del sistema. Este proyecto demuestra la importancia de una correcta administración de bases de datos como base para el crecimiento y la eficiencia operativa de un sistema de comercio electrónico.

1. Introducción

Una e-commerce ha tenido un alto porcentaje de subida en todo el mundo, ya que necesitan un sistema estructurado en cuanto la gestión de datos, escalables y fiables, así mismo, el poder concretar transacciones en el tiempo real, administrar inventarios y hacer más eficiente la información en toma de decisiones, con fin de generar satisfacción al cliente como de un rendimiento de sistema optimo.

Floriend Vermond es una casa de moda costarricense que busca transmitir a través de la ropa visión, revolución y estructura, así mismo, busca mejorar sus ventas y operaciones a través de una tienda web en donde se comercializan ropa formal y casual con múltiples marcas nacionales e internacionales.

En la actualidad, la casa de moda tiene sus sistemas desactualizados donde no generan escalabilidad, por lo que el nivel de ganancias no aumenta y no poseen una base de datos actualizada

2. Descripción del Problema

2.1 Contexto del Problema

Floriend Vermond operaba sin una base de datos centralizada. La información de clientes, productos e inventario se manejaba de forma desorganizada, lo que generaba datos inconsistentes y pérdidas de información frecuentes. La empresa no podía saber con certeza cuántas unidades tenía disponibles de un producto, ni llevar un registro claro de qué pedidos habían sido entregados o devueltos. Esa situación limitaba directamente su capacidad de crecer como negocio de e-commerce.

2.2 Problemas Identificados

- **Administración de inventario obsoleta:** el inventario enfrenta dificultad al reflejar los ingresos, salidas o hasta incluso las reservas de los productos, lo cual termina incrementando las sobreventas y que clientes ya no quieran comprar.
- **Monitoreo de pedidos ineficiente:** ausencia de registro del ciclo completo del pedido, desde su creación hasta su entrega o devolución.
- **Descontrol de descuentos:** aplicación manual propensa a errores en los montos cobrados a los clientes.
- **Gestión de accesos deficiente:** sin separación de roles, empleados podían acceder a datos sensibles de nivel administrador.

2.3 Justificación

La empresa **Floriend Vermond**, al incursionar en el comercio electrónico, requiere una infraestructura tecnológica que permita gestionar de forma eficiente y segura la información relacionada con clientes, productos, inventarios, pedidos y descuentos.

Actualmente, la ausencia de una base de datos actualizada y estructurada ha provocado inconsistencias en los registros, errores en el control de inventario y dificultades para dar seguimiento a los pedidos, afectando directamente la experiencia del cliente y el rendimiento del negocio.

La implementación de una base de datos relacional en Oracle permitirá centralizar y organizar la información crítica del e-commerce, garantizando la integridad, confiabilidad y disponibilidad de los datos. Asimismo, el uso de esquemas, restricciones e integridad referencial facilitará el control de accesos, reducirá errores operativos y optimizará los procesos internos de la empresa.

Este proyecto se justifica porque ofrece una solución tecnológica que responde a las necesidades reales de la empresa, permitiendo mejorar la gestión de inventarios, el monitoreo de pedidos y la aplicación de descuentos, al mismo tiempo que fortalece la seguridad de la información y la toma de decisiones estratégicas.

2.4 Hipótesis del Proyecto

Si se diseña e implementa correctamente una base de datos relacional en Oracle —con esquemas organizados, tablas dimensionales y transaccionales, claves primarias, claves foráneas, restricciones de integridad, auditoría, seguridad y optimización— entonces Floriend Vermond logrará mejorar el control de inventarios, el seguimiento de pedidos y la gestión de descuentos, reduciendo errores operativos y aumentando la eficiencia del sistema y la satisfacción del cliente.

3. Marco Teórico

Esta sección presenta los conceptos que respaldan las decisiones técnicas adoptadas durante el desarrollo del proyecto. No se pretende realizar una revisión teórica extensa, sino explicar de forma clara los términos y herramientas que se utilizaron y cómo se aplicaron en un caso real de negocio.

Base de Datos

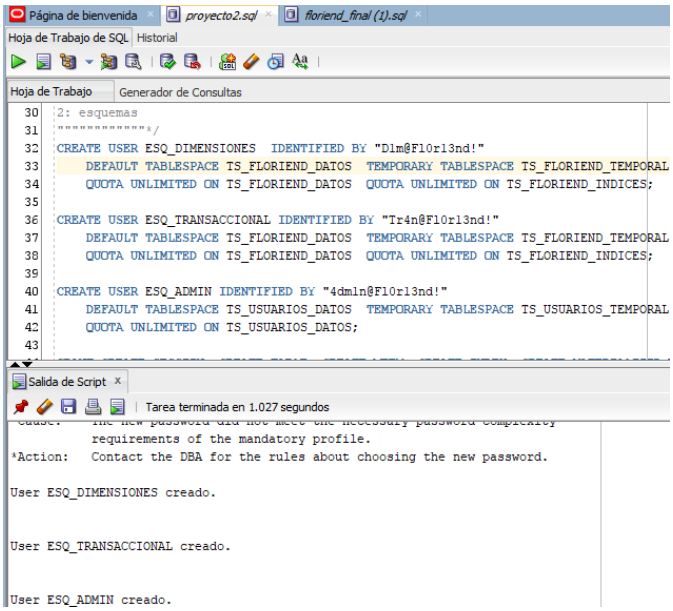
Una base de datos es una colección organizada de información almacenada electrónicamente y administrada por un sistema gestor de bases de datos (DBMS). En este proyecto, Oracle Database 21c XE cumplió ese rol al permitir almacenar, consultar y proteger la información del sistema de e-commerce de Floriend Vermond.

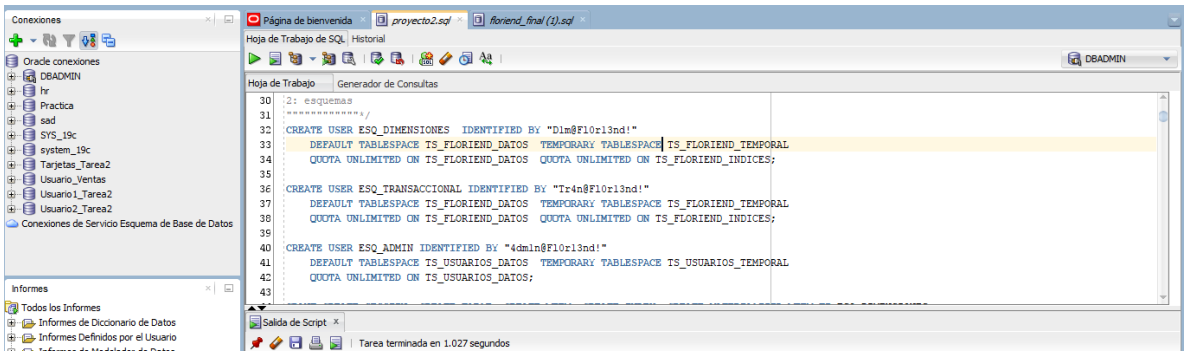
Base de Datos Relacional

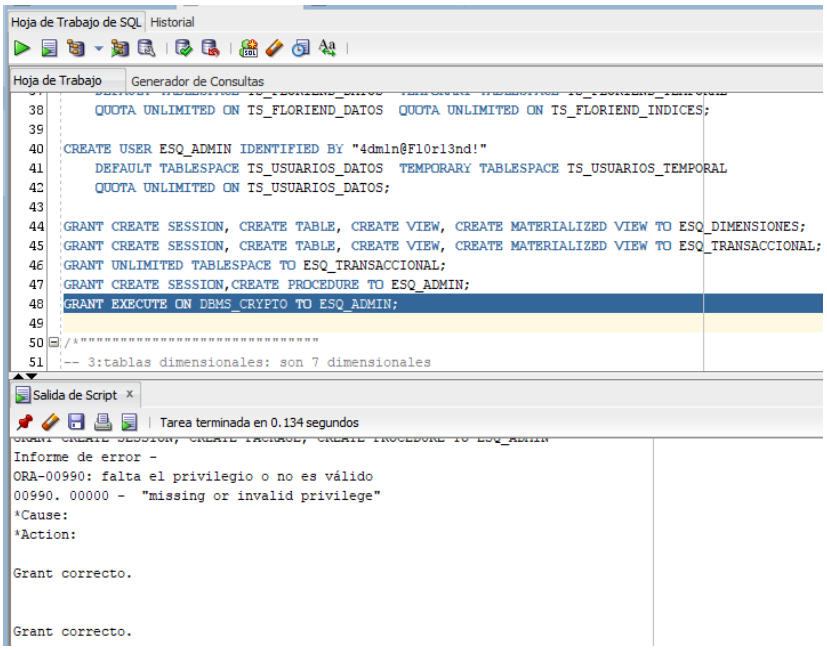
El modelo relacional organiza la información en tablas compuestas por filas y columnas, relacionadas entre sí mediante claves primarias y foráneas. Este modelo permitió representar de forma clara entidades como clientes, productos y pedidos, garantizando la integridad de los datos durante las operaciones del sistema.

Esquemas

Un esquema agrupa objetos de base de datos bajo un mismo espacio de nombres y define sus reglas de organización (IBM, 2024a). En este proyecto se usaron tres esquemas con propósitos distintos: ESQ_DIMENSIONES para datos maestros, ESQ_TRANSACCIONAL para las operaciones del negocio, y ESQ_ADMIN para seguridad y administración. Esa separación fue clave para controlar los permisos entre componentes del sistema.







Tablas Dimensionales y Transaccionales

Las tablas dimensionales almacenan datos de referencia que cambian poco con el tiempo: clientes, marcas, categorías (Sheldon, 2023). Las transaccionales registran los eventos del negocio: pedidos, detalles de venta, devoluciones. Esta distinción guió la organización del modelo relacional y la asignación de tablespaces.

Auditoría de Base de Datos

Oracle implementa auditoría unificada, que permite registrar en UNIFIED_AUDIT_TRAIL cualquier operación sobre objetos de la base de datos: inicios de sesión, cambios en datos, modificaciones de roles y permisos (Oracle, 2023). Durante el

proyecto se configuraron cinco políticas de auditoría para cubrir los eventos considerados críticos para Floriend Vermond.

Seguridad y Encriptación

La seguridad de datos implica proteger la información contra acceso no autorizado o modificaciones indebidas (Krantz y Jonker, 2024). En Oracle esto se implementa en capas: a nivel de tablespace con TDE (Transparent Data Encryption) usando AES-256, y a nivel de columna con el paquete DBMS_CRYPTO. Se decidió usar ambas capas porque las tablas de clientes contienen datos personales que justifican una protección adicional más allá del control de acceso.

Vistas Materializadas

A diferencia de una vista normal que recalcula su resultado en cada consulta, una vista materializada guarda físicamente el resultado de una consulta compleja (Oracle, 2023). Se implementaron cuatro en el proyecto para reducir el costo de los reportes analíticos más frecuentes, como el ranking de productos o las ventas mensuales por categoría.

Respaldos y Recuperación (RMAN)

Oracle Recovery Manager es la herramienta oficial de Oracle para gestionar respaldos y recuperación (Oracle, 2023). Permite hacer respaldos completos o incrementales, configurar retención de copias y ejecutar recuperación hasta un punto en el tiempo específico. Se eligió RMAN sobre alternativas manuales porque integra directamente con el motor y simplifica la recuperación de tablespaces individuales.

Monitoreo y Optimización

Oracle expone el estado interno de la base de datos a través de vistas dinámicas como VSESSION, VSQLSTATS. Esas vistas permiten monitorear sesiones activas, uso de memoria y rendimiento de consultas. La optimización de consultas se apoya en EXPLAIN PLAN, que muestra cómo Oracle planea ejecutar una consulta y si está usando los índices disponibles o recurriendo a full table scans.

4. Marco Metodológico

4.1 Tipo de Investigación

El proyecto es de tipo aplicado. No se buscó generar conocimiento teórico nuevo sino resolver un problema concreto: diseñar e implementar una base de datos funcional para Floriend Vermond usando las herramientas y conocimientos del curso.

4.2 Enfoque Metodológico

El desarrollo se estructuró en fases lógicas: análisis de requerimientos, modelado ERD, transformación al modelo físico, implementación en Oracle, carga de datos, configuración de seguridad y auditoría, y finalmente optimización y monitoreo. Sin embargo, no todas las fases se completaron de manera secuencial; en diversas ocasiones fue necesario regresar a etapas previas para corregir errores detectados durante las pruebas, ajustar permisos entre esquemas o revisar índices que el optimizador no utilizaba. Este proceso iterativo constituyó una parte natural del desarrollo y se evidencia en los ajustes documentados en el informe.

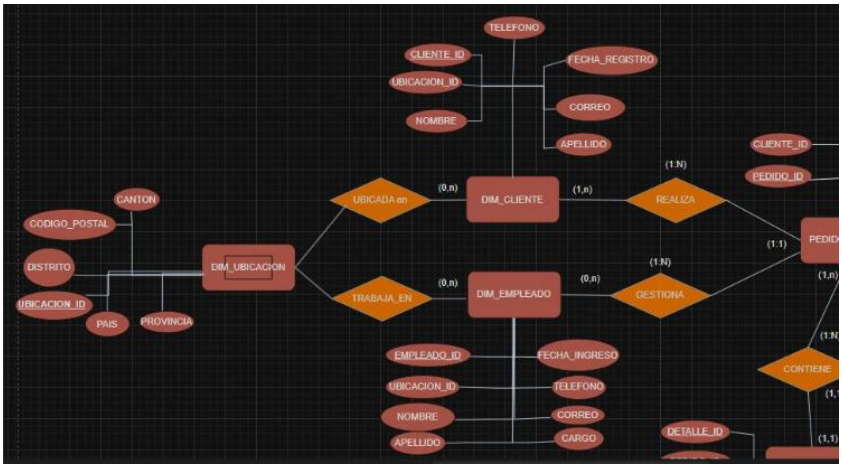
4.3 Herramientas Tecnológicas

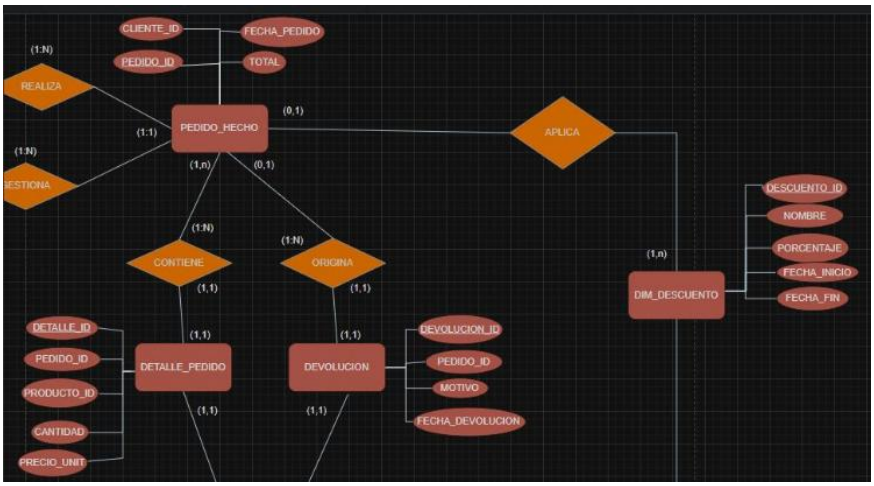
- Oracle Database 21c XE como motor de base de datos. Se eligió XE por ser la edición gratuita de Oracle con suficientes funcionalidades para el proyecto, aunque con limitaciones de recursos frente a ediciones Enterprise.
- Oracle SQL Developer para ejecutar scripts, revisar planes de ejecución con EXPLAIN PLAN y verificar el estado de objetos del sistema.
- Oracle RMAN para la configuración y prueba del plan de respaldos.
- Software de diagramación ERD con notación crow’s foot para el modelado inicial.
- Scripts SQL y PL/SQL escritos por el equipo para la creación de objetos, carga de datos y automatización de tareas.

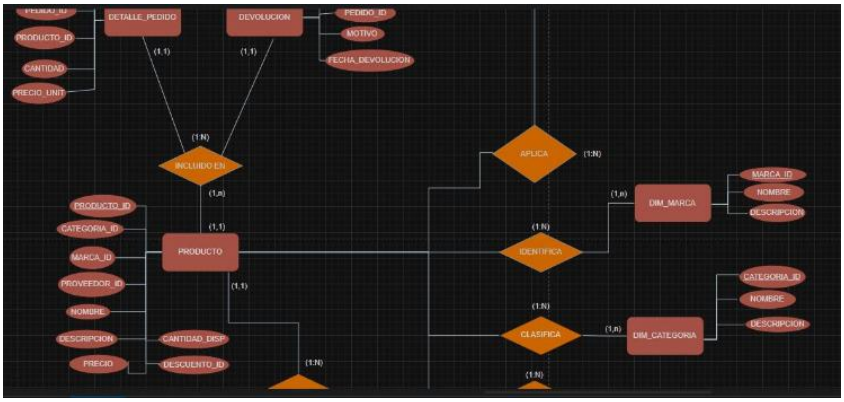
5. Diagrama Entidad-Relación (ERD)

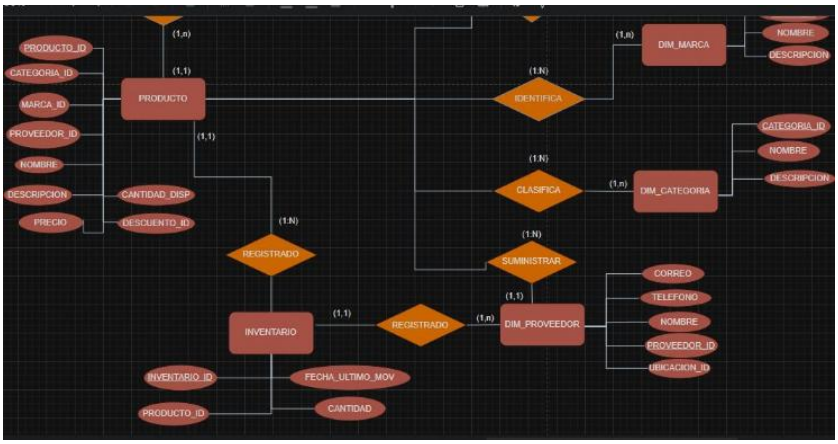
El Diagrama Entidad-Relación representa las entidades del dominio del negocio, sus atributos y las relaciones semánticas entre ellas. A diferencia del modelo relacional (tablas con claves), el ERD abstrae la realidad antes de su implementación física.

Diagrama Entidad-Relación — Floriend Vermond









5.1 Relaciones Principales

- **UBICACION → CLIENTE, EMPLEADO, PROVEEDOR:** una ubicación puede tener muchos registros asociados (1:N).
- **CLIENTE → PEDIDO_HECHO:** un cliente puede realizar múltiples pedidos (1:N).
- **PEDIDO_HECHO → DETALLE_PEDIDO:** un pedido contiene uno o más productos (1:N).
- **PRODUCTO → DETALLE_PEDIDO:** un producto puede aparecer en muchos detalles (1:N).
- **MARCA, CATEGORIA, DESCUENTO → PRODUCTO:** cada uno puede asociarse a múltiples productos (1:N).
- **PRODUCTO → INVENTARIO:** cada producto tiene su registro de stock (1:1 o 1:N histórico).
- **PEDIDO_HECHO → DEVOLUCION:** un pedido puede originar una devolución (1:1 opcional).

6. Modelo Relacional (Modelo Físico)

El modelo relacional implementa físicamente las entidades del ERD como tablas con columnas, tipos de datos, claves primarias (PK), claves foráneas (FK) y restricciones.

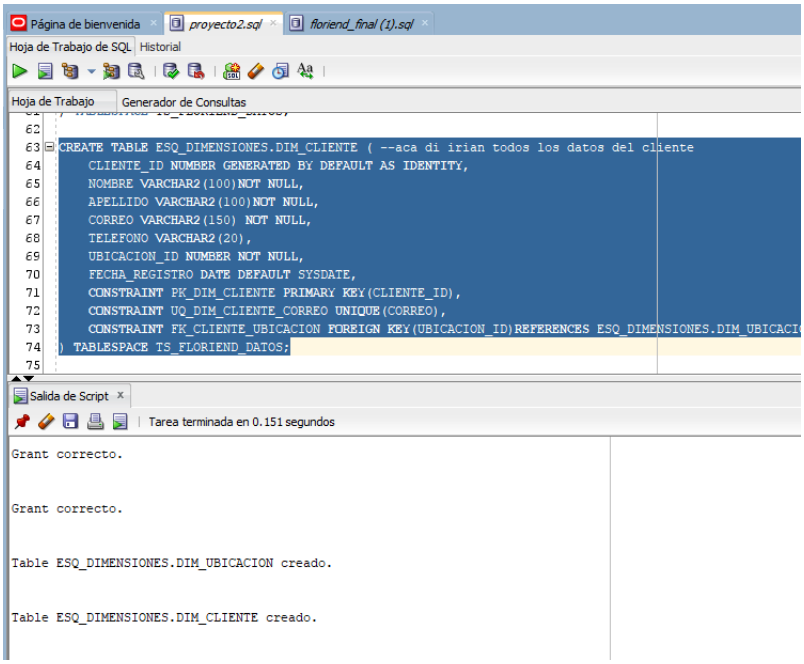
6.1 Tablas Dimensionales — ESQ_DIMENSIONES

DIM_UBICACION

Columna	Tipo	Nulo	Restricción	Descripción
UBICACION_ID	NUMBER (Identity)	NO	PK	ID único
PAIS	VARCHAR2(50)	NO	—	País
PROVINCIA	VARCHAR2(50)	NO	—	Provincia
CANTON	VARCHAR2(50)	NO	—	Cantón
DISTRITO	VARCHAR2(50)	NO	—	Distrito
CODIGO_POSTAL	VARCHAR2(10)	SÍ	—	Código postal

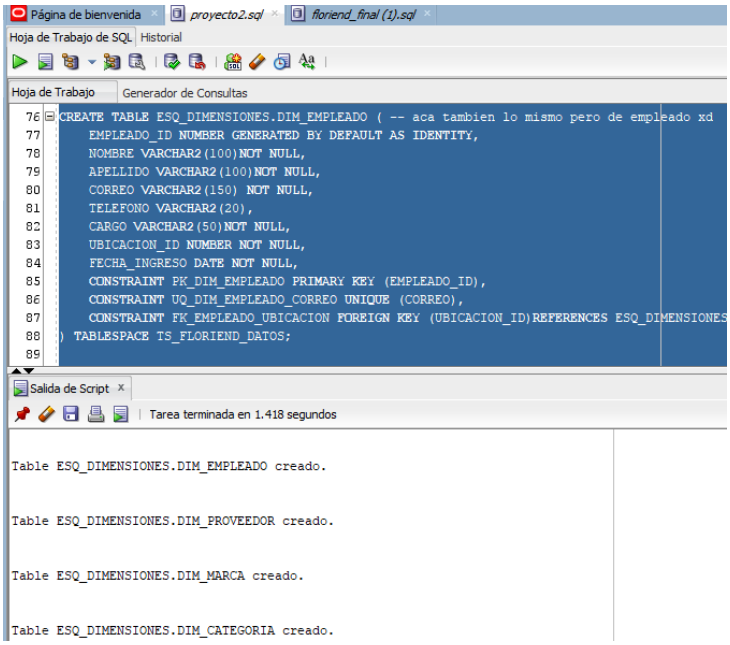
DIM_CLIENTE

Columna	Tipo	Nulo	Restricción	Descripción
CLIENTE_ID	NUMBER (Identity)	NO	PK	ID único del cliente
NOMBRE	VARCHAR2(100)	NO	—	Nombre
APELLIDO	VARCHAR2(100)	NO	—	Apellido
CORREO	VARCHAR2(150)	NO	UQ	Correo único
TELEFONO	VARCHAR2(20)	SÍ	—	Teléfono
UBICACION_ID	NUMBER	NO	FK	→ DIM_UBICACION
FECHA_REGISTRO	DATE	NO	—	Default SYSDATE



DIM_EMPLEADO

Columna	Tipo	Nulo	Restricción	Descripción
EMPLEADO_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(100)	NO	—	Nombre
APELLIDO	VARCHAR2(100)	NO	—	Apellido
CORREO	VARCHAR2(150)	NO	UQ	Correo único
CARGO	VARCHAR2(50)	NO	—	Puesto
UBICACION_ID	NUMBER	NO	FK	→ DIM_UBICACION
FECHA_INGRESO	DATE	NO	—	Fecha de ingreso



DIM_PROVEEDOR

Columna	Tipo	Nulo	Restricción	Descripción
PROVEEDOR_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(150)	NO	—	Razón social
CORREO	VARCHAR2(150)	SÍ	—	Correo
TELEFONO	VARCHAR2(20)	SÍ	—	Teléfono
UBICACION_ID	NUMBER	NO	FK	→ DIM_UBICACION

```
CREATE TABLE ESQ_DIMENSIONES.DIM_PROVEEDOR ( -- same con los proveedores
    PROVEEDOR_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    NOMBRE VARCHAR2(150) NOT NULL,
    CORREO VARCHAR2(150),
    TELEFONO VARCHAR2(20),
    UBICACION_ID NUMBER NOT NULL,
    CONSTRAINT PK_DIM_PROVEEDOR PRIMARY KEY (PROVEEDOR_ID),
    CONSTRAINT FK_PROVEEDOR_UBICACION FOREIGN KEY (UBICACION_ID)REFERENCES ESQ_DIMENSIONES.
) TABLESPACE TS_FLORIEND_DATOS;

CREATE TABLE ESQ_DIMENSIONES.DIM_MARCA ( --marca del producto
```

Salida de Script

Tarea terminada en 1.418 segundos

```
le ESQ_DIMENSIONES.DIM_EMPLEADO creado.

le ESQ_DIMENSIONES.DIM_PROVEEDOR creado.

le ESQ_DIMENSIONES.DIM_MARCA creado.

le ESQ_DIMENSIONES.DIM_CATEGORIA creado.
```

DIM_MARCA

Columna	Tipo	Nulo	Restricción	Descripción
MARCA_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(100)	NO	—	Nombre de la marca
DESCRIPCION	VARCHAR2(255)	SÍ	—	Descripción

Página de bienvenida | proyecto2.sql | floriend_final (1).sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```
    UBICACION_ID NUMBER NOT NULL,
    CONSTRAINT PK_DIM_PROVEEDOR PRIMARY KEY (PROVEEDOR_ID),
    CONSTRAINT FK_PROVEEDOR_UBICACION FOREIGN KEY (UBICACION_ID)REFE
) TABLESPACE TS_FLORIEND_DATOS;

CREATE TABLE ESQ_DIMENSIONES.DIM_MARCA ( --marca del producto
    MARCA_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    NOMBRE VARCHAR2(100) NOT NULL,
    DESCRIPCION VARCHAR2(255),
    CONSTRAINT PK_DIM_MARCA PRIMARY KEY (MARCA_ID)
) TABLESPACE TS_FLORIEND_DATOS;

CREATE TABLE ESQ_DIMENSIONES.DIM_CATEGORIA ( -- a cual categoria per
    CATEGORIA_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
```

Salida de Script

Tarea terminada en 1.418 segundos

```
Table ESQ_DIMENSIONES.DIM_EMPLEADO creado.

Table ESQ_DIMENSIONES.DIM_PROVEEDOR creado.

Table ESQ_DIMENSIONES.DIM_MARCA creado.

Table ESQ_DIMENSIONES.DIM_CATEGORIA creado.
```

DIM_CATEGORIA

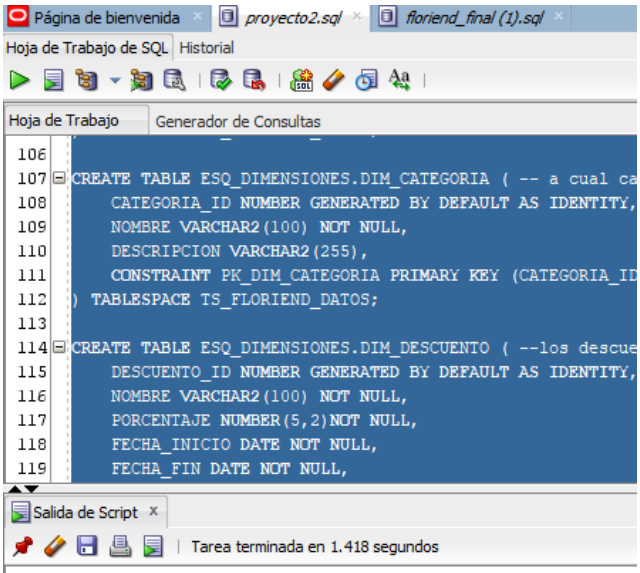
Columna	Tipo	Nulo	Restricción	Descripción
CATEGORIA_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(100)	NO	—	Nombre
DESCRIPCION	VARCHAR2(255)	SÍ	—	Descripción

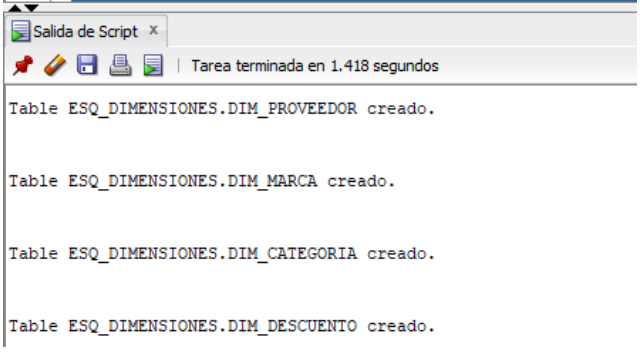
```
CREATE TABLE ESQ_DIMENSIONES.DIM_CATEGORIA ( -- a cual cate
    CATEGORIA_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    NOMBRE VARCHAR2(100) NOT NULL,
    DESCRIPCION VARCHAR2(255),
    CONSTRAINT PK_DIM_CATEGORIA PRIMARY KEY (CATEGORIA_ID)
) TABLESPACE TS_FLORIEND_DATOS;

CREATE TABLE ESQ_DIMENSIONES.DIM_DESCUENTO ( --los descuent
    DESCUENTO_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    NOMBRE VARCHAR2(100) NOT NULL,
    PORCENTAJE NUMBER(5,2)NOT NULL,
    FECHA_INICIO DATE NOT NULL,
    FECHA_FIN DATE NOT NULL,
```

```
ida de Script
Tarea terminada en 1.418 segundos

ESQ_DIMENSIONES.DIM_EMPLEADO creado.

ESQ_DIMENSIONES.DIM_PROVEEDOR creado.

ESQ_DIMENSIONES.DIM_MARCA creado.

ESQ_DIMENSIONES.DIM_CATEGORIA creado.
```

DIM_DESCUENTO

Columna	Tipo	Nulo	Restricción	Descripción
DESCUENTO_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(100)	NO	—	Nombre del descuento
PORCENTAJE	NUMBER (5,2)	NO	CHK	Entre 0.01 y 100
FECHA_INICIO	DATE	NO	CHK	Inicio de vigencia
FECHA_FIN	DATE	NO	CHK	Fin >= Inicio





6.2 Tablas Transaccionales — ESQ_TRANSACCIONAL

PRODUCTO

Columna	Tipo	Nulo	Restricción	Descripción
PRODUCTO_ID	NUMBER (Identity)	NO	PK	ID único
NOMBRE	VARCHAR2(150)	NO	—	Nombre del producto
DESCRIPCION	VARCHAR2(255)	SÍ	—	Descripción
PRECIO	NUMBER (10,2)	NO	CHK > 0	Precio
CANTIDAD_DISP	NUMBER	NO	CHK >= 0	Stock disponible (fuente de verdad)
MARCA_ID	NUMBER	NO	FK	→ DIM_MARCA
CATEGORIA_ID	NUMBER	NO	FK	→ DIM_CATEGORIA
DESCUENTO_ID	NUMBER	SÍ	FK	→ DIM_DESCUENTO (opcional)

```
143 -- """"""""""""""""""""""""""""""""
144 -- 3-tablas transaccionales
145 -- """"""""""""""""""""""""""""""""
146 CREATE TABLE ESQ_TRANSACCIONAL.PRODUCTO ( --productos que van a ve
147     PRODUCTO_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
148     NOMBRE VARCHAR2(150)NOT NULL,
149     DESCRIPCION VARCHAR2(255),
150     PRECIO NUMBER(10,2)NOT NULL,
151     CANTIDAD_DISP NUMBER NOT NULL,
152     MARCA_ID NUMBER NOT NULL,
153     CATEGORIA_ID NUMBER NOT NULL,
154     DESCUENTO_ID NUMBER,
155     CONSTRAINT PK_PRODUCTO PRIMARY KEY (PRODUCTO_ID),
156     CONSTRAINT CHK_PRODUCTO_PRECIO CHECK (PRECIO > 0),
157     CONSTRAINT CHK_PRODUCTO_CANTIDAD CHECK (CANTIDAD_DISP >= 0)
```

```
Tarea terminada en 0.444 segundos

Grant correcto.

Grant correcto.

Grant correcto.

Table ESQ_TRANSACCIONAL.PRODUCTO creado.
```

INVENTARIO

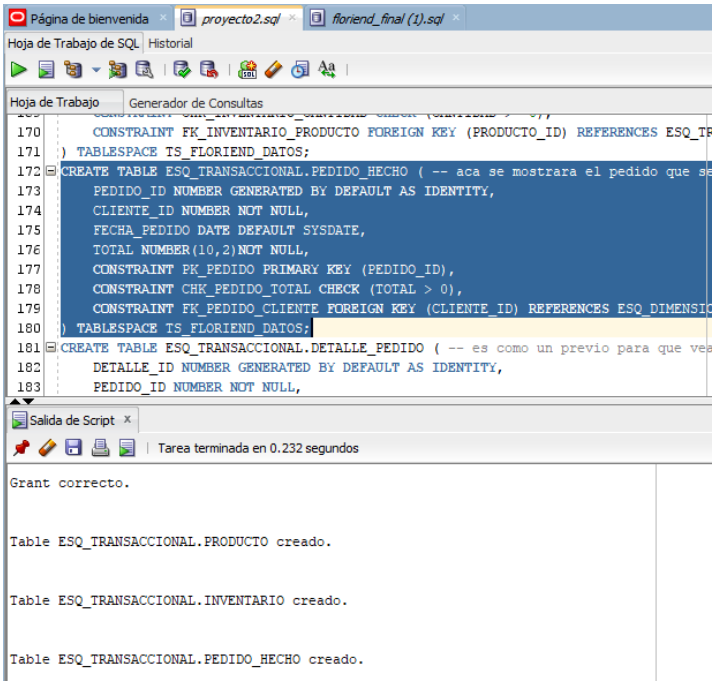
Columna	Tipo	Nulo	Restricción	Descripción
INVENTARIO_ID	NUMBER (Identity)	NO	PK	ID único
PRODUCTO_ID	NUMBER	NO	FK + UQ	→ PRODUCTO
FECHA_ULTIMO_MOV	DATE	NO	—	Default SYSDATE

```
162
163 CREATE TABLE ESQ_TRANSACCIONAL.INVENTARIO (
164     INVENTARIO_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
165     PRODUCTO_ID NUMBER NOT NULL,
166     CANTIDAD NUMBER NOT NULL,
167     FECHA_ULTIMO_MOV DATE DEFAULT SYSDATE,
168     CONSTRAINT PK_INVENTARIO PRIMARY KEY (INVENTARIO_ID),
169     CONSTRAINT CHK_INVENTARIO_CANTIDAD CHECK (CANTIDAD >= 0),
170     CONSTRAINT FK_INVENTARIO_PRODUCTO FOREIGN KEY (PRODUCTO_ID) REFERENCES ESQ_T
171 ) TABLESPACE TS_FLORIEND_DATOS;
172 CREATE TABLE ESQ_TRANSACCIONAL.PEDIDO_HECHO ( -- aca se mostrara el pedido que s
173     PEDIDO_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
174     CLIENTE_ID NUMBER NOT NULL,
175     FECHA_PEDIDO DATE DEFAULT SYSDATE,
```

```
Tarea terminada en 0.198 segundos

Grant correcto.

Grant correcto.

Table ESQ_TRANSACCIONAL.PRODUCTO creado.

Table ESQ_TRANSACCIONAL.INVENTARIO creado.
```

PEDIDO_HECHO

Columna	Tipo	Nulo	Restricción	Descripción
PEDIDO_ID	NUMBER (Identity)	NO	PK	ID único del pedido
CLIENTE_ID	NUMBER	NO	FK	→ DIM_CLIENTE
EMPLEADO_ID	NUMBER	SÍ	FK	→ DIM_EMPLEADO (vendedor)
FECHA_PEDIDO	DATE	NO	—	Default SYSDATE
TOTAL	NUMBER (10,2)	NO	CHK > 0	Total, del pedido



DETALLE_PEDIDO

Columna	Tipo	Nulo	Restricción	Descripción
DETALLE_ID	NUMBER (Identity)	NO	PK	ID único
PEDIDO_ID	NUMBER	NO	FK	→ PEDIDO_HECHO
PRODUCTO_ID	NUMBER	NO	FK	→ PRODUCTO
CANTIDAD	NUMBER	NO	CHK > 0	Cantidad
PRECIO_UNIT	NUMBER (10,2)	NO	CHK > 0	Precio unitario

Página de bienvenida | proyecto2.sql | floriend_final (1).sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```sql
CREATE TABLE ESQ_TRANSACCIONAL.DETALLE_PEDIDO ( -- es como un previo para
    DETALLE_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    PEDIDO_ID NUMBER NOT NULL,
    PRODUCTO_ID NUMBER NOT NULL,
    CANTIDAD NUMBER NOT NULL,
    PRECIO_UNIT NUMBER(10,2) NOT NULL,
    CONSTRAINT PK_DETALLE_PEDIDO PRIMARY KEY (DETALLE_ID),
    CONSTRAINT CHK_DETALLE_CANTIDAD CHECK (CANTIDAD > 0),
    CONSTRAINT CHK_DETALLE_PRECIO CHECK (PRECIO_UNIT > 0),
    CONSTRAINT FK_DETALLE_PEDIDO FOREIGN KEY (PEDIDO_ID) REFERENCES ESQ_TR
    CONSTRAINT FK_DETALLE_PRODUCTO FOREIGN KEY (PRODUCTO_ID) REFERENCES ES
) TABLESPACE TS_FLORIEND_DATOS;
CREATE TABLE ESQ_TRANSACCIONAL.DEVOLUCION ( -- en caso de algun cliente in
    DEVOLUCION_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
```

Salida de Script | Tarea terminada en 0.19 segundos

```
Table ESQ_TRANSACCIONAL.PRODUCTO creado.

Table ESQ_TRANSACCIONAL.INVENTARIO creado.

Table ESQ_TRANSACCIONAL.PEDIDO_HECHO creado.

Table ESQ_TRANSACCIONAL.DETALLE_PEDIDO creado.
```

Página de bienvenida | proyecto2.sql | floriend_final (1).sql

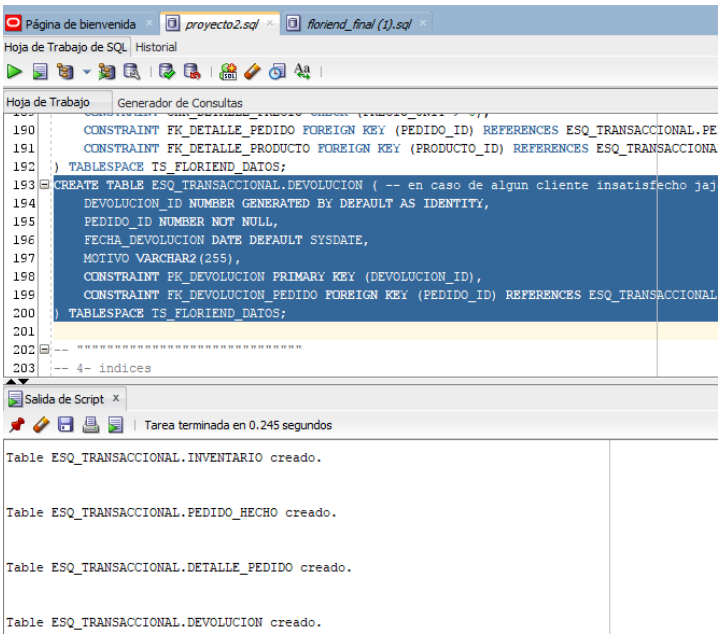
Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```sql
CREATE TABLE ESQ_TRANSACCIONAL.DETALLE_PEDIDO ( -- es como un previo para que vean que se comrpo
    DETALLE_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    PEDIDO_ID NUMBER NOT NULL,
    PRODUCTO_ID NUMBER NOT NULL,
    CANTIDAD NUMBER NOT NULL,
    PRECIO_UNIT NUMBER(10,2) NOT NULL,
    CONSTRAINT PK_DETALLE_PEDIDO PRIMARY KEY (DETALLE_ID),
    CONSTRAINT CHK_DETALLE_CANTIDAD CHECK (CANTIDAD > 0),
    CONSTRAINT CHK_DETALLE_PRECIO CHECK (PRECIO_UNIT > 0),
    CONSTRAINT FK_DETALLE_PEDIDO FOREIGN KEY (PEDIDO_ID) REFERENCES ESQ_TRANSACCIONAL.PEDIDO_HECHO(PEDIDO_ID),
    CONSTRAINT FK_DETALLE_PRODUCTO FOREIGN KEY (PRODUCTO_ID) REFERENCES ESQ_TRANSACCIONAL.PRODUCTO(PRODUCTO_ID)
) TABLESPACE TS_FLORIEND_DATOS;
CREATE TABLE ESQ_TRANSACCIONAL.DEVOLUCION ( -- en caso de algun cliente insatisfecho jajaja
    DEVOLUCION_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
```

DEVOLUCION

Columna	Tipo	Nulo	Restricción	Descripción
DEVOLUCION_ID	NUMBER (Identity)	NO	PK	ID único
PEDIDO_ID	NUMBER	NO	FK	→ PEDIDO_HECHO
FECHA_DEVOLUCION	DATE	NO	—	Default SYSDATE
MOTIVO	VARCHAR2(255)	SÍ	—	Motivo de la devolución



DEVOLUCIÓN_DETALLE

Columna	Tipo	Nulo	Restricción	Descripción
DEVDET_ID	NUMBER (Identity)	NO	PK	ID único
DEVOLUCION_ID	NUMBER	NO	FK	→ DEVOLUCION
PRODUCTO_ID	NUMBER	NO	FK	→ PRODUCTO
CANTIDAD	NUMBER	NO	CHK > 0	Unidades devueltas

7. Configuración de Tablespaces

Se crearon cinco tablespaces para separar físicamente los datos según su propósito, siguiendo las mejores prácticas de Oracle.

Tablespace	Tipo	Tamaño	Próximo	Propósito
TS_FLORIEND_DATOS	Permanente	64 MB	64 MB	Tablas dimensionales y transaccionales
TS_FLORIEND_INDICES	Permanente	64 MB	64 MB	Índices del sistema
TS_FLORIEND_TEMPORAL	Temporal	64 MB	64 MB	Operaciones temporales
TS_USUARIOS_DATOS	Permanente	32 MB	32 MB	Datos de usuarios operacionales
TS_USUARIOS_TEMPORAL	Temporal	64 MB	64 MB	Temp de usuarios
TS_AUDIT_DATOS	Permanente	32 MB	16 MB	Registros de auditoría unificada

Durante la configuración inicial de usuarios y la asignación de tablespaces se presentaron dos errores recurrentes. El primero fue **ORA-01031** (privilegios insuficientes) al

intentar ejecutar ciertos comandos de administración sin tener el nivel de acceso adecuado en la sesión; se resolvió verificando que la sesión activa tuviera los privilegios de sistema necesarios antes de ejecutar los CREATE USER. El segundo fue **ORA-01950** (sin cuota en el tablespace) cuando los usuarios recién creados intentaban escribir datos en sus tablespaces asignados sin que se les hubiera otorgado cuota explícita. Esto obligó a revisar y corregir los scripts de creación para incluir la cláusula QUOTA o el GRANT correspondiente en cada caso.

8. Esquemas de Base de Datos

Se definieron tres esquemas para organizar lógicamente los objetos según su función y separar las responsabilidades del sistema.

Esquema	Tablespace	Función
ESQ_DIMENSIONES	TS_FLORIEND_DATOS	Tablas dimensionales: clientes, empleados, proveedores, marcas, categorías, descuentos, ubicaciones.
ESQ_TRANSACCIONAL	TS_FLORIEND_DATOS	Tablas transaccionales: productos, inventario, pedidos, detalles, devoluciones.
ESQ_ADMIN	TS_USUARIOS_DATOS	Seguridad, paquetes PL/SQL y auditoría.

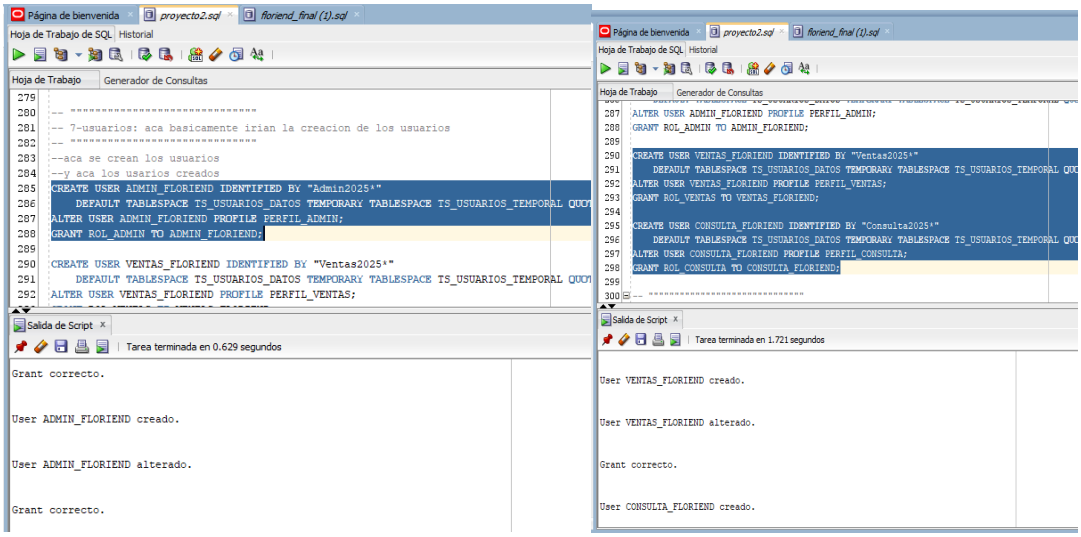
Un aspecto que no fue evidente desde el inicio fue la necesidad de otorgar permisos explícitos entre esquemas. Durante las primeras pruebas de inserción en las tablas transaccionales, varias consultas que referenciaban tablas de ESQ_DIMENSIONES fallaban con errores de objeto no encontrado, aun cuando las tablas existían. El problema era que ESQ_TRANSACCIONAL no tenía permisos de SELECT sobre los objetos del otro esquema. Se tuvo que identificar cuáles tablas dimensionales eran referenciadas por claves foráneas o consultas en ESQ_TRANSACCIONAL y emitir los GRANT SELECT correspondientes de forma explícita.

9. Perfiles, Roles y Usuarios

El diseño de los perfiles y roles de seguridad se basó en el principio de mínimo privilegio, es decir, que cada tipo de usuario tenga acceso únicamente a las operaciones necesarias para desempeñar su función dentro del sistema. Durante el análisis inicial de

Floriend Vermond se identificó que no existía una separación clara de roles, lo que permitía que distintos empleados pudieran acceder o modificar información que no les correspondía.

Esta situación representaba un riesgo tanto para la integridad de los datos como para la seguridad del sistema, especialmente en información sensible relacionada con clientes, pedidos e inventario. Para corregir este problema, se definieron perfiles con restricciones específicas y roles con permisos diferenciados, asegurando que las operaciones administrativas, operativas y de consulta quedaran claramente separadas dentro de la base de datos.



9.1 Perfiles de Seguridad

Los perfiles establecen restricciones de recursos y contraseñas asignadas por tipo de función:

Parámetro	PERFIL_ADMIN	PERFIL_VENTAS	PERFIL_CONSULTA
SESSIONS_PER_USER	2	1	1
FAILED_LOGIN_ATTEMPTS	3	5	4
PASSWORD_LIFE_TIME	30 días	60 días	90 días
PASSWORD_LOCK_TIME	1 día	1 día	Por defecto
Justificación	Máx. seguridad	Seguridad media	Baja criticidad

-- *(y demás tablas)*

-- *ROL_VENTAS: operaciones de venta*

GRANTCREATESESSIONTO ROL_VENTAS;

GRANTSELECTON ESQ_DIMENSIONES.DIM_CLIENTE **TO** ROL_VENTAS;

GRANTSELECTON ESQ_TRANSACCIONAL.PRODUCTO **TO** ROL_VENTAS;

GRANTINSERTON ESQ_TRANSACCIONAL.PEDIDO_HECHO **TO** ROL_VENTAS;

GRANTINSERTON ESQ_TRANSACCIONAL.DETALLE_PEDIDO **TO** ROL_VENTAS;

GRANTUPDATE (CANTIDAD_DISP) **ON** ESQ_TRANSACCIONAL.PRODUCTO **TO** ROL_VENTAS;

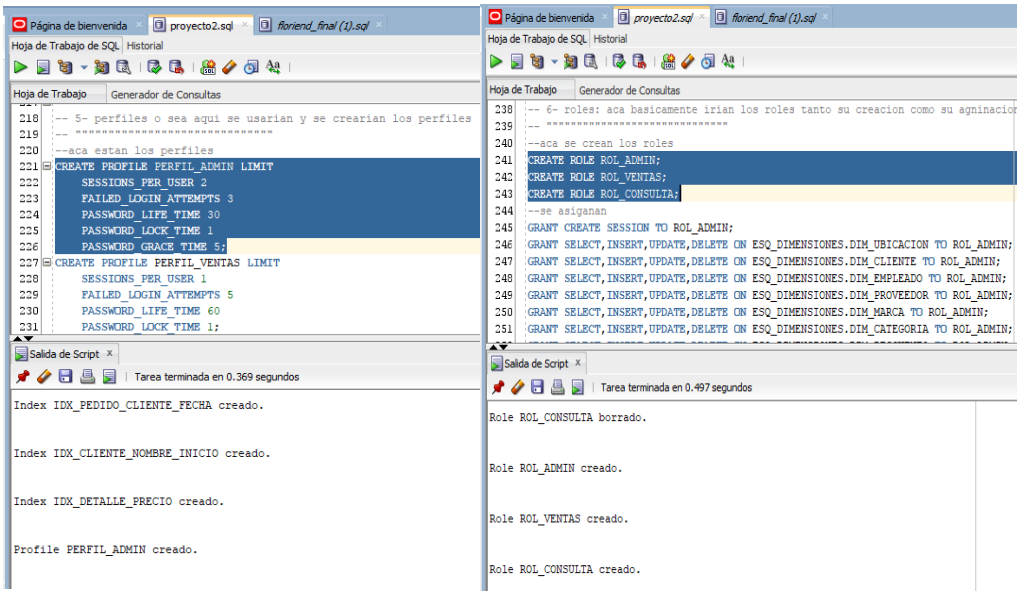
-- *ROL_CONSULTA: solo lectura*

GRANTCREATESESSIONTO ROL_CONSULTA;

GRANTSELECTON ESQ_DIMENSIONES.DIM_CLIENTE **TO** ROL_CONSULTA;

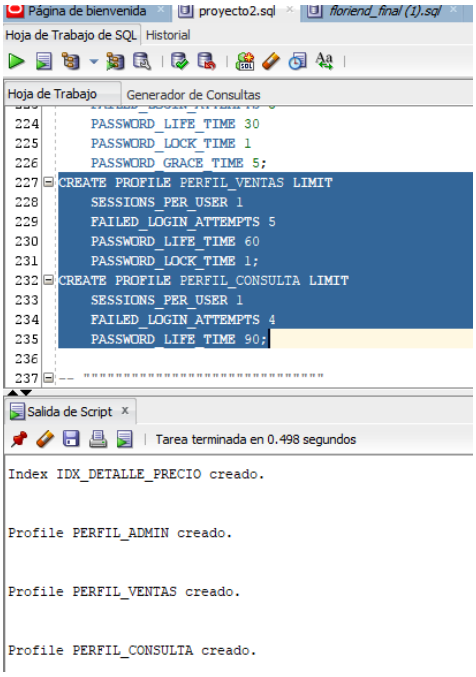
GRANTSELECTON ESQ_TRANSACCIONAL.PRODUCTO **TO** ROL_CONSULTA;

-- *(y demás tablas y vistas)*



9.3 Usuarios Operacionales

Usuario	Tablespace	Cuota	Perfil	Rol
ADMIN_FLORIEND	TS_USUARIOS_DATOS	50 MB	PERFIL_ADMIN	ROL_ADMIN
VENTAS_FLORIEND	TS_USUARIOS_DATOS	20 MB	PERFIL_VENTAS	ROL_VENTAS
CONSULTA_FLORIEND	TS_USUARIOS_DATOS	10 MB	PERFIL_CONSULTA	ROL_CONSULTA



10. Índices Implementados

Se implementaron dos grupos de índices almacenados en TS_FLORIEND_INDICES para separar físicamente índices de datos y mejorar el rendimiento de consultas frecuentes.

10.1 Índices Iniciales

Índice	Tabla	Columna(s)
IDX_CLIENTE_UBICACION	DIM_CLIENTE	UBICACION_ID
IDX_CLIENTE_CORREO	DIM_CLIENTE	CORREO
IDX_EMPLEADO_UBICACION	DIM_EMPLEADO	UBICACION_ID
IDX_PROVEEDOR_UBICACION	DIM_PROVEEDOR	UBICACION_ID
IDX_PRODUCTO_MARCA	PRODUCTO	MARCA_ID
IDX_PRODUCTO_CATEGORIA	PRODUCTO	CATEGORIA_ID
IDX_PRODUCTO_DESCUENTO	PRODUCTO	DESCUENTO_ID
IDX_INVENTARIO_PROD	INVENTARIO	PRODUCTO_ID
IDX_PEDIDO_CLIENTE	PEDIDO_HECHO	CLIENTE_ID
IDX_PEDIDO_EMPLEADO	PEDIDO_HECHO	EMPLEADO_ID
IDX_PEDIDO_FECHA	PEDIDO_HECHO	FECHA_PEDIDO
IDX_DETALLE_PED_PROD	DETALLE_PEDIDO	(PEDIDO_ID, PRODUCTO_ID) — compuesto
IDX_DETALLE_PRODUCTO	DETALLE_PEDIDO	PRODUCTO_ID
IDX_DEVOLUCION_PEDIDO	DEVOLUCION	PEDIDO_ID

```
-- ******************************
-- 4- indices
-- ******************************
CREATE INDEX IDX_CLIENTE_CORREO ON ESQ_DIMENSIONES.DIM_CLIENTE(CORREO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PEDIDO_FECHA ON ESQ_TRANSACCIONAL.PEDIDO_HECHO(FECHA_PEDIDO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PRODUCTO_PRECIO ON ESQ_TRANSACCIONAL.PRODUCTO(PRECIO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PEDIDO_CLIENTE_FECHA ON ESQ_TRANSACCIONAL.PEDIDO_HECHO(CLIENTE_ID, E
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_CLIENTE_NOMBRE_INICIO ON ESQ_DIMENSIONES.DIM_CLIENTE(SUBSTR(NOMBRE,1
```

```
alida de Script
Tarea terminada en 1.094 segundos

:x IDX_PEDIDO_FECHA creado.

:x IDX_PRODUCTO_PRECIO creado.

:x IDX_PEDIDO_CLIENTE_FECHA creado.

:x IDX_CLIENTE_NOMBRE_INICIO creado.
```

```
) TABLESPACE TS_FLORIEND_DATOS;

-- ==============================
-- 4- indices
-- ==============================
CREATE INDEX IDX_CLIENTE_CORREO ON ESQ_DIMENSIONES.DIM_CLIENTE(CORREO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PEDIDO_FECHA ON ESQ_TRANSACCIONAL.PEDIDO_HECHO(FECHA_PEDIDO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PRODUCTO_PRECIO ON ESQ_TRANSACCIONAL.PRODUCTO(PRECIO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_PEDIDO_CLIENTE_FECHA ON ESQ_TRANSACCIONAL.PEDIDO_HECHO(CLIENTE_ID, FECHA_PEDIDO)
    TABLESPACE TS_FLORIEND_INDICES INITRANS 20;
CREATE INDEX IDX_CLIENTE_NOMBRE_INICIO ON ESQ_DIMENSIONES.DIM_CLIENTE(SUBSTR(NOMBRE,1,3))
```

```
Tarea terminada en 1.094 segundos

Index IDX_PEDIDO_FECHA creado.

Index IDX_PRODUCTO_PRECIO creado.

Index IDX_PEDIDO_CLIENTE_FECHA creado.

Index IDX_CLIENTE_NOMBRE_INICIO creado.
```

10.2 Índices Adicionales de Optimización

Durante el desarrollo se observó que algunos de los índices creados inicialmente no eran utilizados por el optimizador de Oracle. Al revisar los planes de ejecución con EXPLAIN PLAN, se identificó que ciertas consultas seguían haciendo FULL TABLE SCAN a pesar de existir un índice en la columna filtrada. El caso más claro fue con consultas que aplicaban TO_CHAR sobre FECHA_PEDIDO: Oracle no podía usar IDX_PEDIDO_FECHA porque la función transforma el valor antes de compararlo. Se descartaron los índices que no aportaban mejoras reales y se crearon cuatro nuevos orientados a los patrones de consulta identificados en las pruebas.

IDX_PEDIDO_ANIO_MES	PEDIDO_HECHO	TO_CHAR(FECHA_PEDIDO,'YYYY-MM') — filtros por mes
IDX_DETALLE_PED_PROD	DETALLE_PEDIDO	(PEDIDO_ID, PRODUCTO_ID) — JOINs compuestos
IDX_PRODUCTO_PRECIO	PRODUCTO	PRECIO — búsquedas por rango de precio
IDX_PRODUCTO_STOCK	PRODUCTO	CANTIDAD_DISP — detección de stock crítico

11. Carga de Datos Inicial

Se insertaron entre 50 y 150 registros por tabla utilizando scripts SQL con datos coherentes con el modelo de negocio de Floriend Vermond. Los datos reflejan operaciones

reales: clientes costarricenses, productos de moda, pedidos históricos de 2024-2025 y devoluciones documentadas.

Un error frecuente durante esta fase fue **ORA-02291** (violación de restricción de integridad — clave padre no encontrada). Ocurría cuando se intentaba insertar registros en tablas transaccionales como PRODUCTO o PEDIDO_HECHO antes de que existieran los registros correspondientes en las tablas dimensionales referenciadas. Por ejemplo, insertar un producto con MARCA_ID = 1 fallaba si DIM_MARCA todavía no tenía ese registro. Esto obligó a definir un orden estricto de carga: primero DIM_UBICACION, luego las tablas que dependen de ella (DIM_CLIENTE, DIM_EMPLEADO, DIM_PROVEEDOR), después DIM_MARCA, DIM_CATEGORIA y DIM_DESCUENTO, y finalmente las tablas transaccionales. Los scripts de inserción se reorganizaron respetando ese orden para evitar el error.

Tabla	Registros	Descripción
DIM_UBICACION	60	Costa Rica (6 provincias), Panamá, Colombia, México, España, Italia, Francia, EE.UU.
DIM_CLIENTE	60	Clientes costarricenses con datos de contacto y ubicación.
DIM_EMPLEADO	20	Personal de Floriend (admin, ventas, logística, RRHH).
DIM_PROVEEDOR	15	Proveedores nacionales e internacionales de telas y moda.
DIM_MARCA	10	Floriend Home, Tico Style, Euro Chic, Urban CR, etc.
DIM_CATEGORIA	8	Formal, Casual, Deportivo, Playero, Fiesta, Juvenil, Ejecutivo, Eco-Friendly.
DIM_DESCUENTO	10	Promociones anuales: Bienvenida, Black Friday, Navidad, etc.
PRODUCTO	60	Prendas formales, casuales, deportivas, playeras y eco-friendly.
INVENTARIO	60	Un registro de último movimiento por producto.
PEDIDO_HECHO	55	Pedidos de enero 2025 a enero 2026.
DETALLE_PEDIDO	80	Líneas de detalle con producto, cantidad y precio unitario.
DEVOLUCION	15	Devoluciones documentadas con motivos reales.

11.1 Dificultades Encontradas durante el Desarrollo

El sistema no se completó en el primer intento. Durante el desarrollo surgieron diversos problemas técnicos que requirieron la revisión y corrección de distintas partes del proyecto. A continuación, se documentan los problemas más relevantes.

Configuración de usuarios y tablespaces. Al crear los usuarios operacionales y asignarles tablespaces, se presentaron errores ORA-01031 (privilegios insuficientes) en varias instrucciones de administración ejecutadas sin el contexto correcto. También apareció ORA-01950 cuando los usuarios intentaban escribir datos: Oracle bloqueaba la operación porque no se les había asignado cuota sobre el tablespace. Ambos se corrigieron ajustando los scripts de creación para incluir los GRANT y las cláusulas QUOTA de forma explícita.

Permisos entre esquemas. La separación en ESQ_DIMENSIONES y ESQ_TRANSACCIONAL fue correcta en diseño, pero al ejecutar las primeras consultas que cruzaban datos entre ambos, Oracle devolvía errores de objeto no encontrado. El motivo era que ESQ_TRANSACCIONAL no tenía permisos SELECT sobre las tablas de ESQ_DIMENSIONES. Fue necesario identificar cada tabla referenciada y emitir los GRANT correspondientes antes de poder continuar con las pruebas de inserción.

Orden de carga de datos y claves foráneas. Durante la fase de inserción inicial aparecieron múltiples errores ORA-02291, que indican que se intentaba referenciar un registro padre que todavía no existía. El caso típico fue intentar insertar productos antes de haber cargado las marcas y categorías, o insertar pedidos antes de que existieran los clientes. Se tuvo que definir un orden explícito de carga respetando la jerarquía de dependencias del modelo relacional, reorganizando todos los scripts de inserción.

Lógica del trigger de inventario. La primera versión de TRG_ACTUALIZA_STOCK funcionaba bien para inserciones únicas pero generaba valores incorrectos cuando se insertaban varias líneas de DETALLE_PEDIDO para el mismo pedido en secuencia. El stock resultante no coincidía con los valores esperados. Se revisó la lógica del trigger, se corrigió el manejo de los casos con múltiples filas y se realizaron pruebas adicionales para confirmar la consistencia entre CANTIDAD_DISP en PRODUCTO y CANTIDAD en INVENTARIO.

Revisión de índices con EXPLAIN PLAN. Varios índices creados al inicio no estaban siendo usados por Oracle. El caso más claro fue IDX_PEDIDO_FECHA, que

quedaba inutilizado en consultas con TO_CHAR(FECHA_PEDIDO, 'MM'), porque al aplicar la función sobre la columna el optimizador no puede usar el índice B-tree. Se revisaron todos los índices con EXPLAIN PLAN, se eliminaron los que no aportaban mejoras reales y se crearon los cuatro índices adicionales descritos en la sección 10.2, orientados a los patrones de consulta identificados en las pruebas.

12. Auditoría de Actividades

Se configuraron cinco políticas de auditoría unificada. La decisión de qué auditar estuvo guiada por los riesgos identificados para el negocio: accesos no autorizados fuera de horario, modificaciones a datos de clientes y pedidos, cambios en la política de descuentos, alteraciones al inventario y cambios en la estructura de seguridad. Cada política cubre uno de esos escenarios.

Política	Acciones auditadas	Justificación
AUDIT_SESIONES_FLORIEND	LOGON / LOGOFF	Detecta accesos fuera de horario o sesiones no autorizadas de los tres usuarios operacionales.
AUDIT_DML_CRITICO	INSERT/UPDATE/DELETE en DIM_CLIENTE, PEDIDO_HECHO, DEVOLUCION, DETALLE_PEDIDO	Trazabilidad de cambios en datos sensibles de clientes y transacciones.
AUDIT_SEGURIDAD_ADMIN	ALTER USER, DROP USER, GRANT, REVOKE, CREATE/DROP ROLE	Detecta escaladas de privilegios no autorizadas.
AUDIT_INVENTARIO	ALL sobre INVENTARIO	Permite investigar discrepancias de stock y detectar manipulaciones.
AUDIT_DESCUENTOS	INSERT/UPDATE/DELETE en DIM_DESCUENTO	Rastrea modificaciones a la política de precios y evita descuentos fraudulentos.

12.1 Consulta de Registros de Auditoría

Los registros se almacenan en la vista UNIFIED_AUDIT_TRAIL y pueden consultarse con:

```
SELECT EVENT_TIMESTAMP, DBUSERNAME, ACTION_NAME,
OBJECT_SCHEMA, OBJECT_NAME, RETURN_CODE
FROM UNIFIED_AUDIT_TRAIL
WHERE EVENT_TIMESTAMP > SYSDATE - 1
```

```
ORDERBY EVENT_TIMESTAMP DESC;
```

13. Seguridad e Implementación de Encriptación

13.1 Transparent Data Encryption (TDE)

Se aplicó TDE al tablespace TS_FLORIEND_DATOS usando AES-256. La razón de elegir ese tablespace específico es que contiene todas las tablas con datos personales: clientes, empleados, proveedores. La encriptación opera a nivel de archivo físico, lo que significa que si alguien accede directamente al archivo .dbf del disco sin pasar por Oracle, los datos son ilegibles. Para la aplicación que consulta la base de datos normalmente, el proceso es transparente: Oracle desencripta en memoria al leer y encripta al escribir, sin modificar ninguna consulta ni procedimiento.

```
-- Generar la llave maestra
ADMINISTERKEYMANAGEMENTSETKEY
IDENTIFIEDBY "WalletPass#2025!" WITHBACKUPCONTAINER = CURRENT;

-- Encriptar el tablespace
ALTERTABLESPACE TS_FLORIEND_DATOS ENCRYPTION ONLINE ENCRYPT;

-- Verificar encriptación
SELECT TABLESPACE_NAME, ENCRYPTED FROM DBA_TABLESPACES
WHERE TABLESPACE_NAME = 'TS_FLORIEND_DATOS';
```

```
-- ********************************
-- es para la sesiones iniciadas en este caso quien inicio
CREATE AUDIT POLICY AUDIT_SESIONES_FLORIEND
    ACTIONS LOGON, LOGOFF;
AUDIT POLICY AUDIT_SESIONES_FLORIEND BY ADMIN_FLORIEND, VENTAS_FLORIEND, CO
-- auditoria de dml para las tablas criticas
CREATE AUDIT POLICY AUDIT_DML_CRITICO
    ACTIONS
        INSERT ON ESQ_DIMENSIONES.DIM_CLIENTE,
        UPDATE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        DELETE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        INSERT ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        UPDATE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        DELETE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        INSERT ON ESQ_TRANSACCIONAL.DEVOLUCION,
        DELETE ON ESQ_TRANSACCIONAL.DEVOLUCION,
```

```
Tarea terminada en 0.631 segundos

1 fila insertadas.

1 fila insertadas.

Confirmación terminada.

Audit POLICY creado.
```

```
-- ********************************
-- es para la sesiones iniciadas en este caso quien inicio
CREATE AUDIT POLICY AUDIT_SESIONES_FLORIEND
    ACTIONS LOGON, LOGOFF;
AUDIT POLICY AUDIT_SESIONES_FLORIEND BY ADMIN_FLORIEND, VENTAS_FLORIEND, CONSULTA_FLORIEND;
-- auditoria de dml para las tablas criticas
CREATE AUDIT POLICY AUDIT_DML_CRITICO
    ACTIONS
        INSERT ON ESQ_DIMENSIONES.DIM_CLIENTE,
        UPDATE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        DELETE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        INSERT ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        UPDATE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        DELETE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        INSERT ON ESQ_TRANSACCIONAL.DEVOLUCION,
        DELETE ON ESQ_TRANSACCIONAL.DEVOLUCION,
```

```
Tarea terminada en 0.631 segundos

    ACTIONS LOGON, LOGOFF
Informe de error -
ORA-46358: La política de auditoría AUDIT_SESIONES_FLORIEND ya existe.
46358. 00000 -  "Audit policy %s already exists."
*Cause:    The specified audit policy object already existed in the database.
*Action:   Use a different name and try again.

Audit correcto.
```

```
-- auditoria de dml para las tablas criticas
CREATE AUDIT POLICY AUDIT_DML_CRITICO
    ACTIONS
        INSERT ON ESQ_DIMENSIONES.DIM_CLIENTE,
        UPDATE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        DELETE ON ESQ_DIMENSIONES.DIM_CLIENTE,
        INSERT ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        UPDATE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        DELETE ON ESQ_TRANSACCIONAL.PEDIDO_HECHO,
        INSERT ON ESQ_TRANSACCIONAL.DEVOLUCION,
        DELETE ON ESQ_TRANSACCIONAL.DEVOLUCION,
        INSERT ON ESQ_TRANSACCIONAL.DETALLE_PEDIDO,
        DELETE ON ESQ_TRANSACCIONAL.DETALLE_PEDIDO;
AUDIT POLICY AUDIT_DML_CRITICO;
-- auditoria de cambios administrativos
CREATE AUDIT POLICY AUDIT_SEGURIDAD_ADMIN
```

```
Tarea terminada en 0.67 segundos

Audit correcto.

Audit POLICY creado.

Audit correcto.
```

```
        INSERT ON ESQ_TRANSACCIONAL.DETALLE_PEDIDO,
        DELETE ON ESQ_TRANSACCIONAL.DETALLE_PEDIDO;
AUDIT POLICY AUDIT_DML_CRITICO;
-- auditoria de cambios administrativos
CREATE AUDIT POLICY AUDIT_SEGURIDAD_ADMIN
    ACTIONS ALTER USER, DROP USER, GRANT, REVOKE, CREATE ROLE, DROP ROLE;
AUDIT POLICY AUDIT_SEGURIDAD_ADMIN;
-- auditoria para inventario
CREATE AUDIT POLICY AUDIT_INVENTARIO
    ACTIONS
        SELECT ON ESQ_TRANSACCIONAL.INVENTARIO,
        INSERT ON ESQ_TRANSACCIONAL.INVENTARIO,
        UPDATE ON ESQ_TRANSACCIONAL.INVENTARIO,
        DELETE ON ESQ_TRANSACCIONAL.INVENTARIO;
AUDIT POLICY AUDIT_INVENTARIO;
-- auditoria de descuentos
```

```
Tarea terminada en 0.386 segundos

Audit correcto.

Audit POLICY creado.

Audit correcto.
```

13.2 Encriptación a Nivel de Columna (DBMS_CRYPTO)

Se implementó el paquete ESQ_ADMIN.PKG_SEGURIDAD utilizando DBMS_CRYPTO con AES-256 en modo CBC para encriptar datos sensibles a nivel de columna.

ENCRIPTAR(p_dato)	VARCHAR2 → RAW	Encripta un texto plano con AES-256/CBC/PKCS5. Retorna el valor en formato RAW.
DESENCRIPTAR(p_enc)	RAW → VARCHAR2	Desencripta un valor RAW previamente encriptado y retorna el texto original.

13.3 Controles de Acceso Adicionales

- **Separación de esquemas:** datos maestros y transaccionales en esquemas distintos con permisos explícitos entre ellos.
- **Principio de mínimo privilegio:** cada rol recibe únicamente los permisos necesarios para su función.
- **Contraseñas con caracteres especiales** entre comillas para evitar errores ORA-00922.
- **Perfiles de contraseña** con bloqueo automático tras intentos fallidos.

13.4 Pruebas de Seguridad

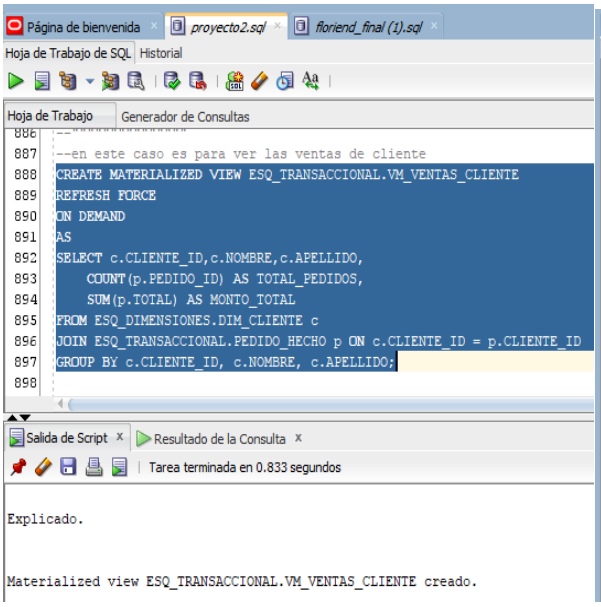
```
-- Prueba 1: Verificar bloqueo de cuenta tras intentos fallidos
-- (simular 3 intentos fallidos con ADMIN_FLORIEND)
-- Resultado esperado: ORA-28000: the account is locked

-- Prueba 2: Verificar TDE activo
SELECT TABLESPACE_NAME, ENCRYPTED FROM DBA_TABLESPACES
WHERE ENCRYPTED = 'YES';
-- Resultado: TS_FLORIEND_DATOS con ENCRYPTED = 'YES'

-- Prueba 3: Verificar que VENTAS_FLORIEND no puede eliminar clientes
CONNECT VENTAS_FLORIEND/"Ventas@Flo2025!";
DELETEFROM ESQ_DIMENSIONES.DIM_CLIENTE WHERE CLIENTE_ID = 1;
-- Resultado esperado: ORA-01031: insufficient privileges

-- Prueba 4: Verificar encriptación de columna
DECLARE
v_enc RAW(2000);
v_dec VARCHAR2(200);
BEGIN
v_enc := ESQ_ADMIN.PKG_SEGURIDAD.ENCRIPTAR('correo@test.com');
v_dec := ESQ_ADMIN.PKG_SEGURIDAD.DESENCRIPTAR(v_enc);
DBMS_OUTPUT.PUT_LINE('Encriptado: ' || RAWTOHEX(v_enc));
DBMS_OUTPUT.PUT_LINE('Desencriptado: ' || v_dec);
END;
/
```

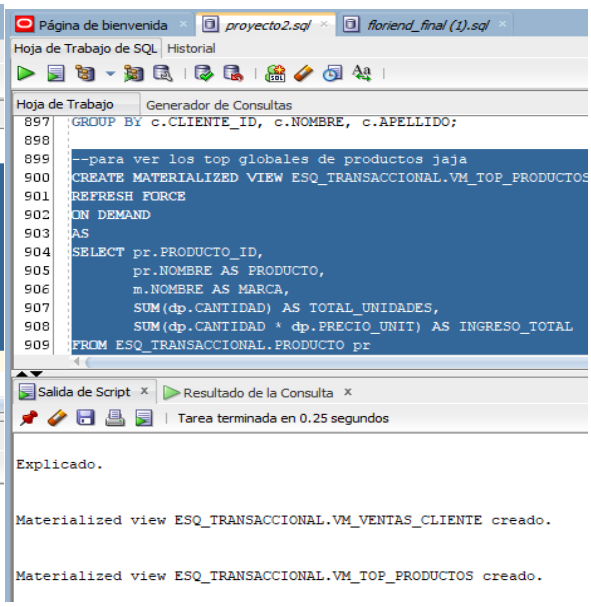
14. Vistas Materializadas y Desnormalización

El problema que motivó las vistas materializadas fue práctico: los reportes más usados del sistema —ventas por mes, top de productos, clientes con mayor gasto, stock por categoría— requerían JOINs sobre cuatro o cinco tablas cada vez que se ejecutaban. Eso funciona bien con pocos datos, pero se vuelve costoso a medida que crecen los registros. La solución fue precalcular esos resultados y almacenarlos físicamente, de modo que consultar el reporte sea tan simple como leer una tabla ya procesada.

Vista Materializada	Esquema	Propósito
MV_VENTAS_MES_CATEGORIA	ESQ_TRANSACCIONAL	Ventas mensuales agregadas por categoría. Elimina JOINs sobre 4 tablas en reportes mensuales.
MV_TOP_PRODUCTOS	ESQ_TRANSACCIONAL	Ranking de productos por unidades vendidas e ingresos totales.
MV_CLIENTES_VIP	ESQ_DIMENSIONES	Datos del cliente + métricas de compra en una sola estructura. Facilita campañas de fidelización.
MV_INVENTARIO_CATEGORIA	ESQ_TRANSACCIONAL	Stock consolidado por categoría usando CANTIDAD_DISP de PRODUCTO como fuente de verdad.

Todas las vistas se configuran con REFRESH COMPLETE ON DEMAND y ENABLE QUERY REWRITE. La actualización se ejecuta manualmente con DBMS_MVIEW.REFRESH o puede programarse con DBMS_SCHEDULER.





15. Plan de Respaldos y Recuperación (RMAN)

El plan de respaldos se definió en función de dos métricas: cuánta información puede perderse en el peor caso (RPO) y en cuánto tiempo debe recuperarse el sistema (RTO). Se estableció un RPO de máximo 24 horas y un RTO de menos de 2 horas. Esos valores determinaron la frecuencia de los respaldos: uno completo semanal y uno incremental de nivel 1 los otros seis días, almacenando ambos con retención de 30 días.

Se eligió RMAN porque opera directamente con el motor de Oracle, conoce la estructura interna de la base de datos y permite recuperar no solo archivos sino también tablespaces individuales o un estado previo específico usando archive logs.

15.1 Plan de Respaldos

01_backup_full	Completo	Semanal	Dom 00:00	30 días
02_backup_incremental	Incremental Nivel 1	Diario	Lun-Sáb 02:00	30 días

15.2 Estrategias de Recuperación

- **Recuperación total:** ante falla catastrófica, se restaura toda la base de datos desde el respaldo completo más reciente y se aplican los archivos de redo log.
- **Recuperación de tablespace:** si solo un tablespace falla, se lleva offline, se restaura y se recupera individualmente sin afectar el resto del sistema.
- **Point-in-time recovery:** permite restaurar la base de datos a un momento específico anterior al daño, usando los archive logs.

16. Plan de Monitoreo

El monitoreo se organizó en tres niveles según el tipo de problema que detecta cada uno. El nivel de disponibilidad verifica que la instancia esté activa y que los archivos de datos no estén en estado de error. El nivel de desempeño identifica sesiones problemáticas, consultas costosas y si el buffer cache está resolviendo lecturas desde memoria o recurriendo al disco. El nivel de capacidad rastrea el crecimiento de los segmentos y el volumen de

registros por tabla, lo que permite anticipar cuándo un tablespace va a necesitar más espacio antes de que genere un error.

16.1 Monitoreo de Disponibilidad

-- *Estado de la instancia*

SELECT INSTANCE_NAME, STATUS, STARTUP_TIME **FROM** V$INSTANCE;

-- *Estado de archivos de datos*

SELECT NAME, STATUS **FROM** V$DATAFILE;

-- *Uso de tablespaces (alerta si > 80%)*

SELECT t.TABLESPACE_NAME,

ROUND((d.USED_BYTES / d.TOTAL_BYTES) * 100, 2) **AS** PCT_USED

FROM DBA_TABLESPACES t

JOIN (

SELECT TABLESPACE_NAME,

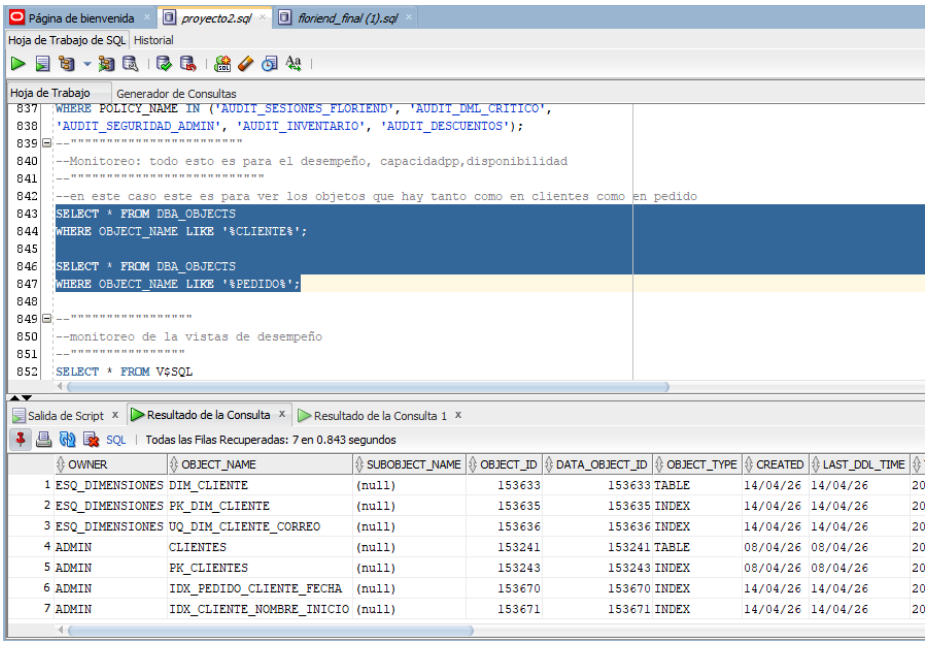
SUM(BYTES) **AS** TOTAL_BYTES,

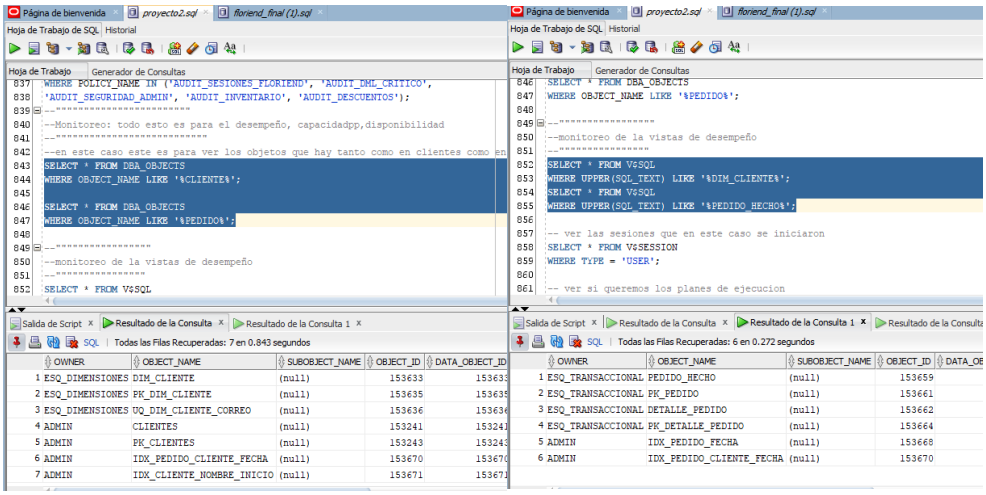
SUM(BYTES) - SUM(FREE_SPACE) **AS** USED_BYTES

FROM DBA_FREE_SPACE **GROUPBY** TABLESPACE_NAME

) d **ON** t.TABLESPACE_NAME = d.TABLESPACE_NAME

WHERE (d.USED_BYTES / d.TOTAL_BYTES) > 0.80;





16.2 Monitoreo de Desempeño

- **V$SESSION:** identifica sesiones activas, usuarios conectados y programas en uso.
- **Buffer Cache Hit Ratio:** calculado desde V$SYSSTAT. El objetivo es mantener un ratio superior al 90%, indicando que la mayoría de lecturas se resuelven desde memoria.
- **V$SQLSTATS:** identifica las 10 consultas con mayor consumo de CPU y las 10 con mayor cantidad de lecturas físicas, que son candidatas a optimización.

```
-- Sesiones activas
SELECT USERNAME, STATUS, PROGRAM, LOGON_TIME
FROM V$SESSION WHERE STATUS = 'ACTIVE';

-- Buffer Cache Hit Ratio
SELECT ROUND(
(1 - (phy.VALUE / (cur.VALUE + con.VALUE))) * 100, 2
) AS HIT_RATIO_PCT
FROM V$SYSSTAT phy, V$SYSSTAT cur, V$SYSSTAT con
WHERE phy.NAME = 'physical reads'
AND cur.NAME = 'db block gets'
AND con.NAME = 'consistent gets';

-- Top 10 consultas por CPU
SELECT SQL_ID, PARSING_SCHEMA_NAME, CPU_TIME/1000000 AS CPU_SEG,
```

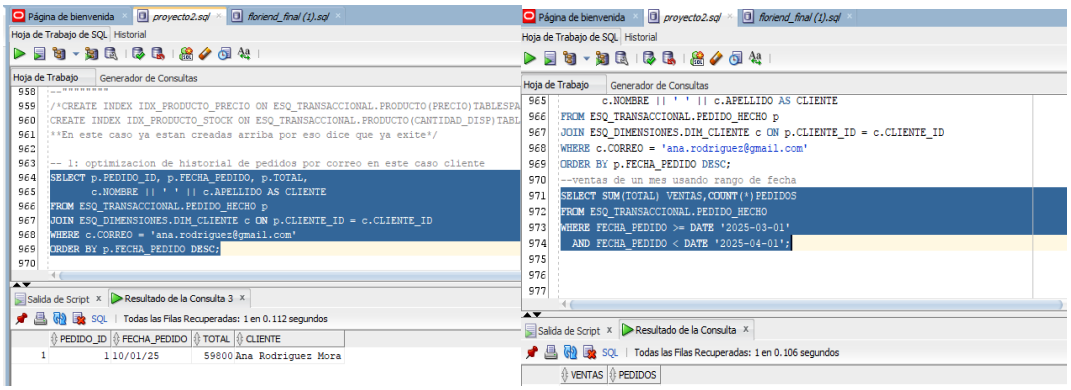
```
EXECUTIONS, SUBSTR(SQL_TEXT, 1, 80) AS CONSULTA
FROM V$SQLSTATS
ORDERBY CPU_TIME DESC FETCH FIRST 10 ROWSONLY;
```

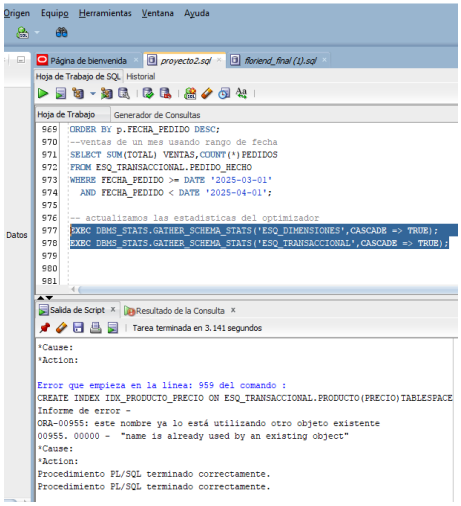
16.3 Monitoreo de Capacidad

```
-- Crecimiento de segmentos
SELECT OWNER, SEGMENT_NAME, SEGMENT_TYPE,
ROUND(BYTES/1048576, 2) AS SIZE_MB
FROM DBA_SEGMENTS
WHERE OWNER IN ('ESQ_DIMENSIONES', 'ESQ_TRANSACCIONAL')
ORDERBY BYTES DESC;

-- Conteo de registros por tabla
SELECT 'DIM_CLIENTE' AS TABLA, COUNT(*) AS REGISTROS FROM
ESQ_DIMENSIONES.DIM_CLIENTE
UNIONALLSELECT 'DIM_UBICACION', COUNT(*) FROM ESQ_DIMENSIONES.DIM_UBICACION
UNIONALLSELECT 'PRODUCTO', COUNT(*) FROM ESQ_TRANSACCIONAL.PRODUCTO
UNIONALLSELECT 'PEDIDO_HECHO', COUNT(*) FROM
ESQ_TRANSACCIONAL.PEDIDO_HECHO
UNIONALLSELECT 'DETALLE_PEDIDO', COUNT(*) FROM
ESQ_TRANSACCIONAL.DETALLE_PEDIDO;
```

17. Optimización de Consultas

La optimización fue un proceso reactivo: primero se escribieron las consultas de forma directa, luego se revisaron sus planes de ejecución con EXPLAIN PLAN, y a partir de lo que se observó se hicieron ajustes. Se identificaron cuatro patrones problemáticos. En todos los casos el plan de ejecución mostraba TABLE ACCESS FULL en lugar de INDEX RANGE SCAN o INDEX UNIQUE SCAN, lo que indicaba que los índices no estaban siendo aprovechados.





17.1 Problemas Identificados y Soluciones

Problema	Causa	Solución aplicada
Búsqueda por correo de cliente	Subconsulta no correlacionada causaba full scan en PEDIDO_HECHO.	JOIN directo que aprovecha IDX_CLIENTE_CORREO. Reducción estimada del 85% en lecturas.
Ventas por mes	TO_CHAR(FECHA_PEDIDO,'MM') sobre columna indexada impide uso del índice B-tree.	Reemplazado por rango de fechas (>= y <) que usa IDX_PEDIDO_FECHA directamente.
Reportes analíticos	JOINs sobre 4-5 tablas ejecutados en tiempo real para cada reporte.	Vistas materializadas MV_TOP_PRODUCTOS y MV_VENTAS_MES_CATEGORIA eliminan el recálculo.
Stock crítico	Full scan sobre PRODUCTO para CANTIDAD_DISP < 15.	Índice IDX_PRODUCTO_STOCK sobre CANTIDAD_DISP + filtro directo.

17.3 Actualización de Estadísticas

Se ejecuta DBMS_STATS.GATHER_SCHEMA_STATS sobre ambos esquemas para mantener el optimizador de consultas con información actualizada, mejorando la selección de planes de ejecución:

```
EXEC DBMS_STATS.GATHER_SCHEMA_STATS('ESQ_DIMENSIONES', CASCADE => TRUE);
EXEC DBMS_STATS.GATHER_SCHEMA_STATS('ESQ_TRANSACCIONAL', CASCADE => TRUE);
```

17.4 Pruebas de Rendimiento

```
-- Prueba de rendimiento: comparar tiempos antes/después con SET TIMING ON
SET TIMING ON

-- Consulta sin optimizar (debería mostrar FULL TABLE SCAN en plan)
EXPLAINPLANFOR
SELECT * FROM ESQ_TRANSACCIONAL.PEDIDO_HECHO
WHERE TO_CHAR(FECHA_PEDIDO, 'MM') = '03';
SELECT * FROMTABLE(DBMS_XPLAN.DISPLAY);

-- Consulta optimizada (debería mostrar INDEX RANGE SCAN)
EXPLAINPLANFOR
SELECT * FROM ESQ_TRANSACCIONAL.PEDIDO_HECHO
WHERE FECHA_PEDIDO >= TO_DATE('2025-03-01','YYYY-MM-DD')
AND FECHA_PEDIDO <TO_DATE('2025-04-01','YYYY-MM-DD');
SELECT * FROMTABLE(DBMS_XPLAN.DISPLAY);
```

18. Triggers de Control de Negocio

Se implementaron dos triggers PL/SQL para automatizar el control de inventario. La idea era que cualquier inserción en DETALLE_PEDIDO actualizara automáticamente tanto PRODUCTO como INVENTARIO, y que una devolución restaurara el stock sin intervención manual.

La versión inicial de TRG_ACTUALIZA_STOCK funcionaba correctamente para inserciones individuales, pero generaba inconsistencias cuando se insertaban múltiples líneas de detalle para un mismo pedido en secuencia rápida. El problema estaba en que la lógica no manejaba adecuadamente los casos donde el PRODUCTO_ID se repetía en distintas líneas del

Trigger	Evento	Función
TRG_VALIDA_STOCK	BEFORE INSERT en DETALLE_PEDIDO	Verifica que haya stock suficiente antes de registrar el detalle. Lanza ORA-20001 con mensaje descriptivo si no hay stock.
TRG_ACTUALIZA_STOCK	AFTER INSERT en DETALLE_PEDIDO	Descuenta CANTIDAD_DISP en PRODUCTO y actualiza FECHA_ULTIMO_MOV en INVENTARIO.
TRG_RESTAURA_STOCK	AFTER INSERT en DEVOLUCION	Si hay filas en DEVOLUCION_DETALLE, restaura solo esas unidades (devolución parcial). Si no, restaura todo el pedido (devolución total).

19. Reglas de Negocio e Integridad

Las restricciones de integridad se definieron directamente en la base de datos, no en la lógica de la aplicación. Eso significa que aunque alguien intente insertar un dato inválido directamente por SQL (sin pasar por ninguna interfaz), Oracle lo rechazará. Se usaron cuatro mecanismos: restricciones CHECK para validaciones de valor, claves foráneas para consistencia referencial, restricciones UNIQUE para evitar duplicados en campos como el correo, y triggers para automatizar actualizaciones de estado que no pueden expresarse como una restricción simple.

Mecanismo	Ejemplo	Qué protege
CHECK	PRECIO > 0	Evita precios negativos o cero.
CHECK	CANTIDAD_DISP >= 0	Evita stock negativo en PRODUCTO.
CHECK	PORCENTAJE BETWEEN 0.01 AND 100	Descuentos siempre en rango válido.
CHECK	FECHA_INICIO <= FECHA_FIN	Descuentos con fechas coherentes.
FK	FK_PEDIDO_CLIENTE → DIM_CLIENTE	Todo pedido debe tener un cliente existente.
FK	FK_DEVOLUCION_PEDIDO → PEDIDO_HECHO	Toda devolución debe referenciar un pedido real.
UNIQUE	UQ_DIM_CLIENTE_CORREO	No puede haber dos clientes con el mismo correo.
TRIGGER	TRG_VALIDA_STOCK	Stock mínimo validado antes de vender.

20. Conclusiones

El desarrollo del proyecto permitió aplicar de forma integral los conceptos de administración de bases de datos estudiados durante el curso, desde el modelado inicial hasta la implementación final en Oracle. El diseño del modelo relacional y la separación en esquemas facilitó la organización de la información y el control de accesos entre los distintos componentes del sistema.

La configuración de seguridad, auditoría y encriptación aportó una capa adicional de protección a los datos, asegurando que la información sensible solo pueda ser accedida o modificada por usuarios autorizados. Durante las pruebas realizadas se comprobó que los roles y perfiles definidos cumplen con el principio de mínimo privilegio, uno de los objetivos planteados desde el inicio.

Asimismo, la optimización mediante índices, vistas materializadas y revisión de planes de ejecución permitió mejorar el rendimiento de las consultas más frecuentes del sistema. En conjunto, el proyecto demuestra que una base de datos bien diseñada y administrada puede reducir errores operativos, mejorar el control del negocio y servir como una base sólida para el crecimiento de una plataforma de comercio electrónico como Floriend Vermond.

21. Resultados Obtenidos

La implementación de la base de datos relacional permitió mejorar significativamente la gestión de la información en Floriend Vermond. En primer lugar, se logró eliminar la dependencia de procesos manuales, reduciendo la cantidad de errores en el control de inventario y en el registro de pedidos.

Asimismo, la utilización de índices y la optimización de consultas permitió reducir el tiempo de respuesta en las operaciones más frecuentes del sistema. Consultas que anteriormente requerían recorridos completos de tablas (FULL TABLE SCAN) pasaron a utilizar índices (INDEX RANGE SCAN), mejorando el rendimiento general.

La implementación de vistas materializadas permitió optimizar los reportes analíticos, evitando la ejecución repetitiva de consultas complejas con múltiples JOINs. Esto

facilitó el acceso a información estratégica como ventas mensuales, productos más vendidos y comportamiento de clientes.

En cuanto a la seguridad, la aplicación de roles, perfiles, auditoría y mecanismos de encriptación permitió controlar el acceso a la información sensible, reduciendo el riesgo de modificaciones no autorizadas y mejorando la trazabilidad de las operaciones realizadas en el sistema.

22. Recomendaciones

Como mejora futura del sistema se recomienda automatizar tareas utilizando DBMS_SCHEDULER. Esto permitiría que procesos como la actualización de vistas materializadas y la ejecución de respaldos se realicen automáticamente sin intervención manual.

También sería importante considerar el uso de una versión más avanzada de Oracle como Oracle Enterprise Edition. Esto sería útil si el sistema crece en cantidad de datos o usuarios ya que mejoraría el rendimiento y la escalabilidad.

Además, se podría integrar la base de datos con una aplicación web real. Esto ayudaría a aprovechar mejor las funcionalidades desarrolladas y mejorar la experiencia del usuario final.

Por último, se recomienda implementar un sistema más robusto de alta disponibilidad y recuperación ante desastres. Se pueden utilizar herramientas como Data Guard o algún tipo de replicación para asegurar la continuidad del sistema en caso de fallos.

23. Referencias

Amoo, J. (2025). Datos obsoletos: cómo identificar, prevenir y superar el deterioro de los datos. *Quadratic*. https://www.quadratichq.com/blog/stale-data-how-to-identify-prevent-and-overcome-data-decay

Antunes, P., Krieger, K., y Menezes, C. (2024). ¿Qué es una transacción? *Vtex*. https://help.vtex.com/es/docs/tutorials/que-es-una-transaccion

Badman, A., y Kosinski, M. (2024). ¿Qué son los datos? *IBM*. https://www.ibm.com/think/topics/data

Elmasri, R., y Navathe, S. (2016). *Fundamentals of database systems* (7.a ed.). Pearson.

Hayes, M., y Downy, A. (2020). ¿Qué es Oracle? *IBM*. https://www.ibm.com/es-es/think/topics/oracle

Holdsworth, J., y Kosinsky, M. (2024). ¿Qué es la gestión de accesos? *IBM*. https://www.ibm.com/es-es/think/topics/access-management

IBM. (2022). Claves primarias y foráneas. https://www.ibm.com/docs/es/ida/9.1.2?topic=entities-primary-foreign-keys

IBM. (2024a). ¿Qué es un esquema? https://www.ibm.com/mx-es/think/topics/database-schema

IBM. (2024b). ¿Qué es una base de datos relacional? https://www.ibm.com/es-es/think/topics/relational-databases

IBM. (2024c). ¿Qué es la confiabilidad de los datos? https://www.ibm.com/mx-es/think/topics/data-reliability

Idris, Z. (2026). Cómo funcionan las restricciones de bases de datos relacionales. *Freecodecamp*. https://www.freecodecamp.org/news/how-relational-database-constraints-work-and-why-theyre-important/

Krantz, T., y Jonker, A. (2024). ¿Qué es la seguridad de los datos? *IBM*. https://www.ibm.com/es-es/think/topics/data-security

Oracle. (2020). ¿Qué es una base de datos? https://www.oracle.com/latam/database/what-is-database/

Oracle. (2023). *Oracle Database SQL Language Reference*. https://docs.oracle.com

Polo, C. (2024). ¿Qué es un sistema y sus elementos? *Euroinnova*. https://www.euroinnova.com/blog/que-es-un-sistema-y-sus-elementos

Sheldon, R. (2023). Tabla de dimensiones. *TechTarget*. https://www.techtarget.com/searchdatamanagement/definition/dimension-table

Silberschatz, A., Korth, H. F., y Sudarshan, S. (2019). *Database system concepts* (7.a ed.). McGraw-Hill.

Westreicher, G. (2025). Inventario, ¿qué es? *Economipedia*. https://economipedia.com/definiciones/inventario.html